

TSOpt guide 4: Major genes, haplotypes and fixed effects

Contents

1	Why genome-wide criteria cannot see major genes	1
2	Finding major loci with <code>tso_major_scan()</code>	2
3	Marker effects you already have: <code>tso_major_effects()</code>	4
4	Haplotypes with <code>tso_haplotypes()</code>	6
5	Genome-wide haplotypes: blocks, kernel and coverage	6
6	Protecting alleles: random, fixed or a constraint	8
7	The introgression demonstration	9
7.1	When it does not matter	11
8	The allele and haplotype coverage report	11
9	Weighted kinship with <code>tso_kinship(weights =)</code>	12
10	General fixed effects with <code>tso_design(fixed =)</code>	12
11	Summary checklist	13
12	References	13

This guide covers the parts of a genomic prediction model that a genome-wide relationship matrix does not see: major genes, the haplotypes around them, and fixed effects such as trials, checks and covariates. It shows how to find major loci, how to make sure the training set can learn their effects, and how TSOpt handles any fixed effect exactly.

```
library(TSOpt)
```

1 Why genome-wide criteria cannot see major genes

CDmean, PEVmean, the r-score and D-optimality all look at the genome through one relationship matrix K . The VanRaden (2008) matrix is a sum over markers, and each marker contributes in proportion to its variance $2p(1 - p)$. With thousands of markers, one marker is a tiny share of K .

Take a panel of 400 doubled-haploid lines and a trait in which one QTL explains 40% of the genetic variance:

```
panel <- tso_simulate_panel(n = 400, markers = 1500, groups = 4, fst = 0.25, seed = 8)
g <- tso_qc(panel, verbose = FALSE)
```

```
tr <- tso_simulate_trait(panel, h2 = 0.6, n_qtl = 80, major = 1,
                        major_share = 0.4, seed = 2)
qtl <- attr(tr, "major")
p <- colMeans(g$dosage) / 2
weight <- 2 * p * (1 - p) / sum(2 * p * (1 - p))
c(qtl_weight = weight[[qtl]], average_weight = mean(weight))
#>      qtl_weight average_weight
#> 0.0004994177 0.0007097232
```

The QTL, m1128, carries 0.05% of the weight in K . That is no more than an average marker (0.07%): the matrix cannot tell it from the other 1408. Removing it from the matrix altogether changes almost nothing:

```
K <- tso_kinship(g)
K_without <- tso_kinship(g, weights = setNames(0, qtl))
cor(K[upper.tri(K)], K_without[upper.tri(K)])
#> [1] 0.9999958
```

A criterion built on K therefore has no reason to care whether the training set contains both alleles of the QTL. If one allele is missing, the model cannot estimate the gene's effect, and every target carrying that allele is predicted from its polygenic background alone, however good the CDmean looks.

What to look at: a major gene is 0.05% of the relationship matrix but 40% of the genetic variance. The criterion sees the first number, the field sees the second.

2 Finding major loci with `tso_major_scan()`

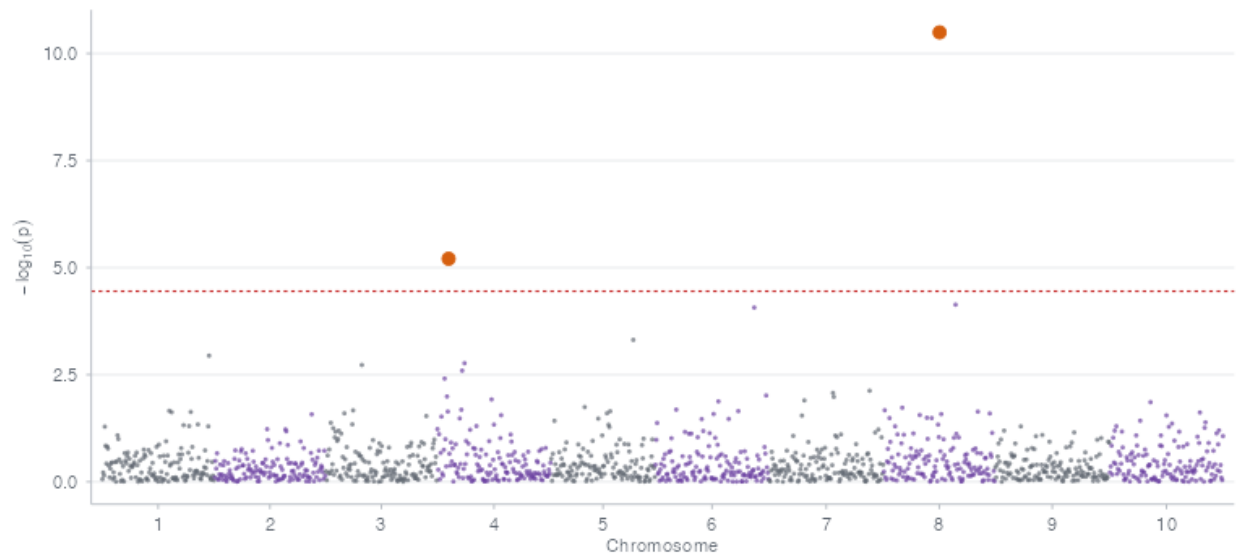
Usually you do not know the loci in advance. `tso_major_scan()` runs a fast mixed-model genome scan (EMMAX, Kang et al. 2010) on earlier phenotypes: one REML fit of the polygenic model, then a generalised least-squares test of every marker.

```
scan <- tso_major_scan(panel, setNames(tr$y, tr$id), verbose = FALSE)
scan
#> TSOpt major-locus scan · 400 lines · 1,409 markers · mixed model (EMMAX) (h2 = 0.59)
#> Threshold -log10(p) >= 4.4
#> Major loci 2: m1128 (chr 8, -log10 p 10.5, 34% of genetic variance); m466 (chr 4,
  ↪ -log10 p 5.2, 12% of genetic variance)
#> Next      tso_data(geno, major = scan) (uses a 47% variance share at these loci)
```

```
tso_plot(scan)
```

2 major locus/loci found

mixed model (EMMAX) · 400 lines



The planted QTL was m1128, and the scan found it. The other lead, m466, was not planted as a major gene: it is a polygenic QTL of larger effect, or chance. That is the nature of a screen. How to read the output:

- **Leads.** After the most significant marker is taken, markers on the same chromosome within window markers (default 50) of it, or in LD with it ($r^2 > \text{clump_r2}$, default 0.5), are not reported as separate loci. This is clumping: one locus, one lead.
- **Threshold.** The default is Bonferroni at 5% on the $-\log_{10} p$ scale, here 4.4. Change it with `threshold =`.
- **Variance share.** Marginal shares absorb linked polygenic effects and the winner's curse, so the scan refits all leads jointly and reports `share_joint`: the share of genetic variance each lead explains in a model that also contains the polygenic background. The total, `scan$major_share` (here 47%, against 40% planted), is what `tso_data(major = scan)` will use. Pass `major_share = yourself` if you trust another estimate.

```
cols <- c("marker", "chrom", "maf", "effect", "log10p", "share_joint")
knitr::kable(scan$leads[, cols], digits = 3)
```

marker	chrom	maf	effect	log10p	share_joint
m1128	8	0.157	0.815	10.493	0.337
m466	4	0.275	-0.405	5.208	0.115

A lax threshold picks up polygenic noise as well:

```
lax <- tso_major_scan(panel, setNames(tr$y, tr$id), threshold = 3, verbose = FALSE)
lax$leads$marker
#> [1] "m1128" "m466" "m881" "m717"
```

The extra leads explain little each (see `lax$leads$share_joint`). Protecting them costs training slots and buys little, so keep the default threshold unless you have a reason.

What to look at: the scan is a screen for design, not a GWAS. Confirm any locus with your own association analysis, several environments and ideally a known gene, before you build a breeding decision on it.

3 Marker effects you already have: `tso_major_effects()`

Often the loci are known before any phenotype of the new candidates exists: from a published GWAS or QTL study, from last year's BayesB model, or from a cloned gene with a diagnostic marker. `tso_major_effects()` takes those effects instead of phenotypes and returns the same kind of object as the scan, so everything that follows (protection, the plan's Major line) works the same way.

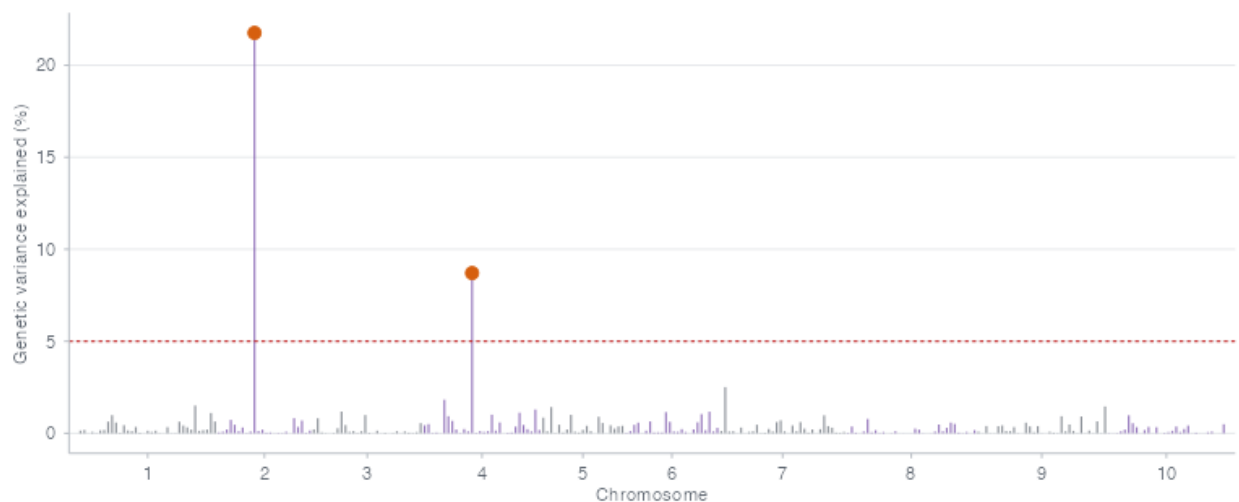
The example panel ships with such a file: effects from a genome-wide model for a disease-resistance score, with two major genes on a polygenic background. As in many published tables, one gene is reported for the other allele.

```
g_ex <- tso_read(system.file("extdata", "example_panel.vcf.gz", package = "TSOpt"),
  verbose = FALSE)
eff <- read.csv(system.file("extdata", "example_marker_effects.csv", package =
  "TSOpt"))
head(eff, 3)
#>   marker chrom   pos effect_allele  effect
#> 1     m5     1 5e+05             A -0.013330
#> 2     m7     1 7e+05             A -0.014700
#> 3     m9     1 9e+05             A  0.002271
me <- tso_major_effects(g_ex, eff)
#> TSOpt major loci from marker effects · 300 lines · 290 of 290 markers matched (1
  ↪ effect alleles flipped)
#> Variance  variance of the genomic values the effects predict here (0.120)
#> Major loci 2: m146 (chr 2, MAF 0.20, 22% of genetic variance); m348 (chr 4, MAF
  ↪ 0.23, 9% of genetic variance)
#> Next      tso_data(geno, major = effects) (uses a 31% variance share at these loci)
```

`tso_plot(me)`

2 major locus/loci from the marker effects

290 marker effects · 300 lines



```
knitr::kable(me$leads[, c("marker", "chrom", "maf", "effect", "share_genetic",
  ↪ "share_region")], digits = 3)
```

marker	chrom	maf	effect	share_genetic	share_region
m146	2	0.203	0.200	0.218	0.218
m348	4	0.227	-0.122	0.087	0.087

How it works:

- **Matching.** Markers are matched by name to your genotypes. Effect alleles (column allele, effect_allele or a1) are matched to the alleles in your file, and an effect given for the other allele is flipped (1 here). Markers you have not genotyped, or that QC removed, are reported and ignored.
- **Shares in your panel.** A marker's contribution is its effect squared times its dosage variance *among your candidates*. A gene with a large effect whose favourable allele is rare in your material gets a small share, as it should.
- **Relative to what?** The genetic variance is var_g if you give it, otherwise $h^2 \times \text{var}_p$, otherwise (for a genome-wide model) the variance of the genomic values the effects predict in your panel. For a short list of genes and none of these, the loci are protected with the default share of 30%.
- **Which kind of effects?** type = "joint" is for one model fitted on all markers (BayesB, BayesC, BayesR, LASSO, known genes). If such a model splits one QTL over several markers in LD, share_region adds them up. type = "marginal" is for GWAS, one marker at a time. Marginal effects overlap through LD, so they are never summed; only significant markers (p_max, Bonferroni by default) are candidates, and the genetic variance must be given. A file with a p-value column is read as GWAS unless you say otherwise. GWAS effects from a small study are inflated by the winner's curse (the Beavis effect), so their shares are upper bounds; effects from a large published study are better.

GWAS summary statistics work the same way. Here the scan's own table stands in for a published GWAS:

```
gw <- data.frame(snp = scan$table$marker, beta = scan$table$effect, pval =
  ↪ 10^-scan$table$log10p)
tso_major_effects(panel, gw, h2 = 0.6, var_p = var(tr$y))
#> TSOpt major loci from marker effects · 400 lines · 1,409 of 1,409 markers matched
#> Variance h2 x phenotypic variance (0.985)
#> Major loci 2: m1128 (chr 8, MAF 0.16, 36% of genetic variance); m466 (chr 4, MAF
  ↪ 0.28, 13% of genetic variance)
#> Next tso_data(geno, major = effects) (uses a 51% variance share at these loci)
```

Ridge effects cannot size a major gene. rrBLUP and GBLUP shrink every effect towards zero by the same amount and spread a major gene over thousands of markers. In simulations, a QTL explaining 30% of the genetic variance received well under 1% in its ridge effect. TSOpt recognises ridge effects (dense, near-normal) and warns. Use GWAS or BayesB effects, run tso_major_scan() on the phenotypes, or name the known loci with major = c("m146", "m348") and major_share =.

The result goes to tso_data() like a scan. Each locus is weighted in the major-locus kernel by the share it explains, so the 25% gene counts more than the 10% one:

```
tp_me <- tso_data(g_ex, major = me, verbose = FALSE)
tp_me$major$loci
#> [1] "m146" "m348"
round(tp_me$major$share, 2)
#> [1] 0.31
plan_me <- tso_design(tp_me, n = 40, verbose = FALSE)
knitr::kable(plan_me$major[, c("class", "freq_candidates", "carriers_training",
  ↪ "required")], digits = 2)
```

class	freq_candidates	carriers_training	required
m146:A	0.20	8	2
m146:C	0.80	32	2
m348:A	0.23	10	2

class	freq_candidates	carriers_training	required
m348:C	0.77	30	2

A data frame of effects can also go straight to `tso_data(major = eff)`.

4 Haplotypes with `tso_haplotypes()`

A single marker may tag several haplotypes at a major locus, and only some of them may carry the favourable allele of the causal gene. `tso_haplotypes()` builds haplotype alleles from runs of adjacent markers, either around chosen markers or tiled along the genome:

```
hp <- tso_haplotypes(panel, around = scan$leads$marker, window = 2)
hp
#> TSOpt haplotypes · 2 block(s), fixed 5-marker tiles · 400 lines · homozygous lines:
  ↳ strings are haplotypes
#> markers per block: median 5 (range 5-5) · haplotypes per block: median 16.5
#> hap_m1128      5 markers · 13 haplotypes (10 with freq >= 2%) · top 30%
#> hap_m466       5 markers · 20 haplotypes (15 with freq >= 2%) · top 12%
round(head(hp$freq[[1]], 6), 3)
#>
#> 01111 01101 01100 00111 00101 11000
#> 0.300 0.195 0.142 0.072 0.068 0.058
```

`window = 2` gives 5-marker blocks (two markers on each side of the lead). Without `around`, the whole genome is tiled into blocks of the same size:

```
tiles <- tso_haplotypes(panel)
length(tiles$blocks)
#> [1] 300
```

For inbred lines and doubled haploids the multi-locus string is the haplotype, as the printout says. For heterozygous material (outcrossers, clones, F2s), a **phased** VCF (genotypes written 0|1) keeps one haplotype per chromosome copy: `tso_read()` and `tso_qc()` carry them along, and a line then carries a haplotype if either copy has it. Unphased data give the local multi-locus genotype instead: still a useful class for protecting alleles, but not a haplotype. Phase the file first (Beagle 5, SHAPEIT or Eagle) if you need true haplotypes; TSOpt does not phase.

Haplotype loci can be protected exactly like markers, by passing the object to `tso_data(major = hp)`, or both at once with `major = list(markers = "m700", haplotypes = hp)`.

5 Genome-wide haplotypes: blocks, kernel and coverage

Haplotypes are not only for major genes. With blocks along the whole genome they give three more tools.

Blocks. `method = "fixed"` tiles the genome into blocks of $2 * \text{window} + 1$ markers, `method = "window"` into windows of kb kilobases, and `method = "ld"` joins adjacent markers while their mean r^2 with the block stays above $r2$ (default 0.3), so blocks follow the LD of your panel. You can also read blocks made by plink `--blocks (blocks = "file.blocks.det")`.

```
hp_ld <- tso_haplotypes(panel, method = "ld")
hp_fx <- tso_haplotypes(panel, method = "fixed", window = 2)
c(ld_blocks = length(hp_ld$blocks), ld_median_markers =
  ↳ stats::median(hp_ld$info$markers),
```

```
fixed_blocks = length(hp_fx$blocks))
#>          ld_blocks ld_median_markers    fixed_blocks
#>          1493          1          300
```

LD blocks are only as long as LD itself. In this simulated panel, adjacent markers are hardly correlated, so most LD blocks are single markers; fixed tiles still carry haplotype identity (which founder a segment comes from) because every line is a mosaic of a few founders.

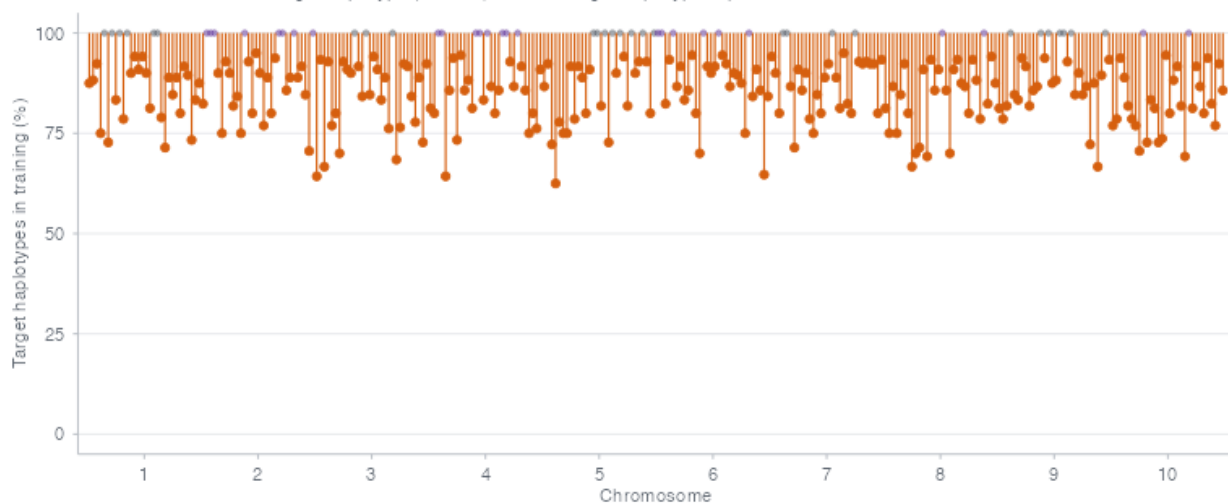
Coverage in every plan. Give the blocks to `tso_data(haplotypes = )` and every plan reports how many of the targets' haplotypes are in the training set, how many target haplotype copies that covers, and how many haplotypes have at least two training carriers (one is not enough to estimate an effect):

```
tgt <- rownames(panel$dosage)[seq(3, 400, length.out = 60)] # 60 targets across the
  <- panel
tp_h <- tso_data(panel, haplotypes = hp_fx, verbose = FALSE)
plan_h0 <- tso_design(tp_h, n = 40, target = tgt, verbose = FALSE)
plan_h1 <- tso_design(tp_h, n = 40, target = tgt, criterion = "haplotype", verbose =
  <- FALSE)
round(rbind(default = unlist(plan_h0$alleles$haplotypes[c("frac", "frac_copies",
  <- "frac2")])),
      haplotype = unlist(plan_h1$alleles$haplotypes[c("frac", "frac_copies",
  <- "frac2")])), 3)
#>          frac frac_copies frac2
#> default  0.868      0.956 0.638
#> haplotype 0.892      0.965 0.636
```

```
tso_plot(plan_h0, "haplotypes")
```

87% of the targets' haplotypes are in the training set

300 blocks · 246 blocks miss a target haplotype (marked) · 96% of target haplotype copies covered · 64% with ≥ 2 carriers



The haplotype criterion. `criterion = "haplotype"` chooses the set that covers the targets' haplotypes, weighted by how many target copies carry each, and each extra carrier of a haplotype counts half the previous one, so a second carrier is still rewarded. The objective is submodular, so the greedy choice is within $1 - 1/e$ of the best possible. It respects every constraint (quotas, checks, major-gene carriers). `tso_pareto(objectives = c("cdmean", "haplotypes"))` shows the trade-off with CDmean.

The haplotype relationship matrix. `tso_kernels(which = "HAP")` treats every block as a multi-allelic locus whose alleles are its haplotypes, and builds the relationship matrix from

the centred haplotype dosages, as VanRaden does for SNPs. With single-marker blocks it is exactly the VanRaden matrix; longer blocks add identity by descent. It is an additive kernel, so it combines with A (kernel = c("A", "HAP"), robust shares) and works with value = "breeding".

Does it pay? On four real panels with real phenotypes (maize, rice, sorghum, switchgrass; 120 paired runs, benchmark/HAPLOTYPES.md), designing under the haplotype kernel predicted no better than the SNP kernel (-0.006 to +0.001 in predictive ability). The haplotype criterion covered 3-4 more percentage points of the targets' haplotypes in every run, and gave more of them a second carrier, at no average cost in prediction, but it predicted better in maize (+0.05) and worse in rice and sorghum (-0.02 to -0.04). So keep the default for prediction, and use the haplotype criterion, or a Pareto front of both, when haplotype representation is the goal: genebank cores, reference panels, rare haplotypes that must be learnable.

6 Protecting alleles: random, fixed or a constraint

`tso_data(major = )` accepts marker names, a scan, a haplotype object, or a list of markers and haplotypes. It then combines two tools:

1. **A model for the locus.** With `major_model = "random"` (the default) the relationship matrix becomes $(1-s)K_{poly} + sK_{major}$, where s is `major_share`. CDmean then rewards sets that sample the locus's alleles. With `major_model = "fixed"` the loci enter the model as fixed effects, as Bernardo (2014) recommends for known large-effect genes, and the design optimises the polygenic CDmean with the loci projected out exactly.
2. **A requirement.** Every allele class that the targets carry must appear at least `min_carriers` times in the training set (default 2). This is a hard constraint, honoured by the greedy, exchange and tabu searches.

Situation	Use	Call
Locus found by a scan, share estimated	random model	<code>tso_data(geno, major = scan)</code>
Known markers, share roughly known	random model	<code>major = c("m1", "m2"), major_share = 0.3</code>
Known gene you will fit as fixed at prediction	fixed model	<code>major_model = "fixed"</code>
One or two genes, each over 10% of variance	fixed model	<code>major_model = "fixed"</code>
Only need carriers, keep the CDmean as is	constraint only	<code>major_share = 0, min_carriers = 3</code>
Rare allele, very few candidate carriers	raise the constraint	<code>min_carriers = 3 or more</code>

Two rules of thumb. First, **design with the model you will predict with**: if you will fit the gene as a fixed effect, design with `major_model = "fixed"`. Second, the constraint is what guarantees carriers; the model only makes the criterion value them. The fixed-model CDmean measures the polygenic part only, so its value is not comparable with a random-model or standard CDmean.

7 The introgression demonstration

A resistance gene has been introgressed into a few lines from distant germplasm. It is carried by 16 of the 60 targets (group G4) and by only 8 candidates, all from the unrelated group G1. This is the scenario of the package benchmark (benchmark/major_genes_benchmark.R), built the same way:

```
panel <- tso_simulate_panel(n = 400, markers = 1500, groups = 4, fst = 0.25, seed = 8)
ids <- rownames(panel$dosage)
targets <- ids[panel$groups == "G4"][1:60]
panel$dosage[, "m700"] <- 0
panel$dosage[sample(targets, 16), "m700"] <- 2
panel$dosage[sample(ids[panel$groups == "G1"], 8), "m700"] <- 2
```

Four training sets of 30 lines: the standard design, the two models, and the constraint alone.

```
tp <- tso_data(panel, verbose = FALSE)
tp_rnd <- tso_data(panel, major = "m700", major_share = 0.3, min_carriers = 3,
  verbose = FALSE)
tp_fix <- tso_data(panel, major = "m700", major_model = "fixed", min_carriers = 3,
  verbose = FALSE)
tp_con <- tso_data(panel, major = "m700", major_share = 0, min_carriers = 3,
  verbose = FALSE)
data_sets <- list(standard = tp, random = tp_rnd, fixed = tp_fix, constraint = tp_con)
plans <- lapply(data_sets, tso_design, n = 30, target = targets, verbose = FALSE)
carriers <- sapply(plans, function(p) sum(panel$dosage[p$selected$id, "m700"] > 0))
carriers
#> standard random fixed constraint
#> 0 3 3 3
```

The standard design contains 0 carriers. It is built from G4 lines, the targets' relatives, which is exactly right for the polygenic background and exactly wrong for the gene. The three protected designs contain 3, 3, 3: exactly the requirement, no more.

```
plans$random
#> TSOpt plan · inbred · 30 lines of 340 candidates · 60 named targets
#> Criterion CDmean, exact (fixed intercept) (averaged over h2 = 0.2, 0.5, 0.8)
  ↳ Search greedy (robust CDmean: average over h2) (0.0 s)
#> Expected CDmean 0.566 (at h2 = 0.50) (exact, intercept projected: 0.544) vs
  ↳ random 0.326 ± 0.149 (better than 100% of 100 random sets)
#> Verdict GREEN optimisation pays off here: expected accuracy ~35.8% above random
#> Structure discrete groups (4 groups, silhouette 0.61) -> stratified allocation
  ↳ 0/0/0/30 (major-allele carriers take precedence)
#> Coverage nearest training relative >= 0.20 for 100% of targets · 0 targets
  ↳ unreachable (< 0.05)
#> Divide 0 candidates effectively unselectable (reliability < 0.06) · see
  ↳ tso_plot(plan, "divide")
#> Major 1 loci · 2 of 2 allele classes carried by targets have >= 3 training
  ↳ carriers
#> Alleles 100.0% of target alleles present in training (0 markers not covered) ·
  ↳ 95.5% of target 5-marker haplotypes
#> Plot value dropping one plot costs 0.34% to 3.75% of the CDmean (median 0.48%) · see
  ↳ tso_plot(plan, "value")
#> Next tso_plot(plan) · tso_export(plan, "trial.csv") · tso_explain(plan)
```

```
tso_plot(plans$random, "major")
```

Major-locus alleles in the training set

1 loci - 2 of 2 allele classes protected



What to look at: the Major line of the printout and the figure. Both allele classes carried by the targets have at least 3 training carriers (the tick marks the requirement). A class shown as *not learnable* has no carrier: its effect cannot be estimated from this training set.

Does it matter for accuracy? We simulate traits in which the gene explains 40% of the genetic variance, and score each training set with the model a breeder would use once the gene is known: GBLUP with the gene as a fixed effect. When the training set has no carrier, the gene's effect cannot be estimated and is dropped.

```
Kp <- tso_kinship(tso_qc(panel, verbose = FALSE))
x_gene <- panel$dosage[, "m700"]
predict_gene <- function(train, y, h2 = 0.5) {
  S <- match(train, ids)
  X <- cbind(1, x_gene)
  if (var(x_gene[S]) == 0) X <- X[, 1, drop = FALSE]
  Vi <- solve(Kp[S, S] + diag((1 - h2) / h2, length(S)))
  XS <- X[S, , drop = FALSE]
  b <- solve(crossprod(XS, Vi %*% XS), crossprod(XS, Vi %*% y[train]))
  setNames(as.vector(X %*% b + Kp[, S] %*% (Vi %*% (y[train] - XS %*% b))), ids)
}
accuracy <- function(plans, eval, n_traits = 10) {
  t(sapply(seq_len(n_traits), function(s) {
    tr <- tso_simulate_trait(panel, h2 = 0.5, n_qtl = 100, major = "m700",
                             major_share = 0.4, seed = s)
    y <- setNames(tr$y, tr$id); gv <- setNames(tr$g, tr$id)
    sapply(plans, function(p) cor(predict_gene(p$selected$id, y)[eval], gv[eval]))
  }))
}
acc <- accuracy(plans, targets)
knitr::kable(data.frame(design = names(plans), carriers = carriers,
                        accuracy = colMeans(acc),
                        better_than_standard = colSums(acc > acc[, "standard"])),
              digits = 2, row.names = FALSE)
```

design	carriers	accuracy	better_than_standard
standard	0	0.22	0
random	3	0.89	10
fixed	3	0.89	10
constraint	3	0.89	10

Over these 10 traits, accuracy on the targets rose from 0.22 with the standard design to 0.89 with the gene protected; in this example the three ways of protecting it do equally well. The full benchmark (30 traits, Rscript benchmark/major_genes_benchmark.R) gives **0.23 to 0.87** for this targeted design and **0.31 to 0.69** for an untargeted one, with the protected design better in 30 of 30 traits in both.

7.1 When it does not matter

Protection is not magic, and it is not always needed. Suppose the gene's other carriers are relatives of the targets: 12 more G4 lines instead of 8 lines from G1.

```
panel_rel <- panel
panel_rel$dosage[, "m700"] <- 0
panel_rel$dosage[targets[x_gene[targets] > 0], "m700"] <- 2
set.seed(4)
g4_rest <- setdiff(ids[panel$groups == "G4"], targets)
panel_rel$dosage[sample(g4_rest, 12), "m700"] <- 2
std_rel <- tso_design(tso_data(panel_rel, verbose = FALSE), n = 30,
                     target = targets, verbose = FALSE)
maj_rel <- tso_design(tso_data(panel_rel, major = "m700", min_carriers = 3,
                              verbose = FALSE), n = 30, target = targets,
                     verbose = FALSE)
c(standard = sum(panel_rel$dosage[std_rel$selected$id, "m700"] > 0),
   major_aware = sum(panel_rel$dosage[maj_rel$selected$id, "m700"] > 0),
   lines_in_common = length(intersect(std_rel$selected$id, maj_rel$selected$id)))
#>      standard      major_aware lines_in_common
#>          11          12          26
```

The standard CDmean already picks carriers, because they are the targets' close relatives. The major-aware design is almost the same set. Protection matters when the carriers sit **away** from the targets in genetic space: introgressions, exotic donors, a rare allele in a small or untargeted set.

8 The allele and haplotype coverage report

Every plan, with or without major =, reports how much of the targets' genetic variation the training set contains. It is the Alleles line of the printout and plan\$alleles:

```
al <- plans$standard$alleles
data.frame(allele_coverage = round(al$frac, 4), markers_not_covered =
  ↪ al$n_missing_markers,
           haplotype_coverage = round(al$haplotypes$frac, 3),
           haplotype_blocks = al$haplotypes$n_blocks)
#>   allele_coverage markers_not_covered haplotype_coverage haplotype_blocks
#> 1      0.9992          2      0.954          285
head(al$missing)
#> [1] "m146" "m700"
```

- frac: of the alleles carried by at least one target, the share also carried by a training line.
- n_missing_markers and missing: the markers where a target allele is absent from training. Here m700 is on the list: the report flags the introgression even though the standard design was blind to it.
- haplotypes\$frac: the same share for 5-marker haplotypes tiled along the genome (n_blocks blocks). Haplotype coverage is usually lower than allele coverage, because rare combinations are many.

What to look at: a few missing markers are normal. A known gene in the missing list, or haplotype coverage far below that of a comparable plan, is a reason to add major = and design again.

9 Weighted kinship with `tso_kinship(weights = )`

`tso_kinship()` accepts marker weights, $G = ZWZ' / (k \sum w p(1 - p))$. Markers without a weight get 1. Two uses: up-weight markers near known QTL, for example with the scan's $-\log_{10} p$, or give weight 0 to drop markers that you model separately.

```
g_intro <- tso_qc(panel, verbose = FALSE)
nudge <- sapply(c(100, 1000, 10000), function(w) {
  Kw <- tso_kinship(g_intro, weights = c(m700 = w))
  p <- tso_design(tso_data(panel, kinship = Kw, verbose = FALSE), n = 30,
    target = targets, verbose = FALSE)
  sum(panel$dosage[p$selected$id, "m700"] > 0)
})
setNames(nudge, c("w = 100", "w = 1000", "w = 10000"))
#>      w = 100   w = 1000   w = 10000
#>         0         0         8
```

A weight is a blunt tool: the design ignores the gene until the marker counts 10,000 times as much as any other, and then takes all 8 carriers at once. Use weights to shape the polygenic kernel, and major = with min_carriers when you need a guarantee.

10 General fixed effects with `tso_design(fixed = )`

Genomic prediction models usually contain more than an intercept: trial or block effects, checks, covariates such as flowering date. These use up information, so a good design should account for them. `fixed =` takes names of columns of `data$units` (factors become indicator columns) or a numeric matrix with one row per unit, or per line with row names.

```
tp$units$trial <- rep(c("T1", "T2", "T3"), length.out = nrow(tp$units))
tp$units$block <- rep(paste0("B", 1:4), each = 100)
plan_trial <- tso_design(tp, n = 40, fixed = c("trial", "block"), verbose = FALSE)
plan_trial$spec$fixed
#> [1] "intercept + 5 fixed column(s) (exact)"
plan_trial$spec$fixed_columns
#> [1] "trial:T2" "trial:T3" "block:B2" "block:B3" "block:B4"
```

A check indicator is just another column, for example `tp$units$check <- tp$units$id %in% my_checks` and `fixed = "check"`. A covariate matrix works the same way:

```
set.seed(5)
covariates <- cbind(flowering = rnorm(400, 60, 4), height = rnorm(400, 95, 8))
rownames(covariates) <- ids
plan_cov <- tso_design(tp, n = 40, fixed = covariates, verbose = FALSE)
plan_cov$spec$fixed
#> [1] "intercept + 2 fixed column(s) (exact)"
```

In multi-environment data the environment means are added automatically; you do not need to ask for them (multi-trait data likewise get one mean per trait, or per trait x environment, see guide 3):

```
met <- tso_data(panel, envs = c("E1", "E2", "E3"), verbose = FALSE)
plan_met <- tso_design(met, n = 90, target = targets, min_per_env = 20,
                      max_per_line = 1, verbose = FALSE)
plan_met$spec$fixed_columns
#> [1] "env:E2" "env:E3"
```

Fixed major loci (major_model = "fixed") are added in the same way. The printout shows the fixed effects in its Criterion line:

```
out <- capture.output(print(plan_trial))
cat(sub(" +[(]averaged.*", "", grep("^Criterion", out, value = TRUE)), "\n")
#> Criterion CDmean, exact (intercept + 5 fixed effects)
```

Why it is exact. A fixed effect is a random effect with an infinitely flat prior. TSOpt runs its fast rank-one search on $K + cXX'$ with a very large constant c , while the contrasts it scores (the targets' genetic values) still come from K . That gives the exact prediction error of the genetic values with X estimated by generalised least squares, the "exact" CDmean of Rincet et al. (2012) extended to any fixed effects, at no extra cost per step. The same device gives the CDmean with block effects fixed that tso_layout() reports for a field layout (guide 3).

11 Summary checklist

- Run tso_major_scan() on earlier phenotypes of the same trait. No lead means a polygenic trait: the genome-wide kernel is enough.
- No phenotypes, but marker effects from a GWAS, a QTL study or a BayesB model? Use tso_major_effects(). Not ridge (rrBLUP / GBLUP) effects: they cannot size a major gene.
- Confirm leads with your own GWAS or known genes before relying on them.
- Consider tso_haplotypes() around each lead when one marker may tag several haplotypes.
- Pass the loci to tso_data(major =). Choose major_model = "fixed" if you will fit them as fixed effects at prediction, otherwise the random model with the scan's share.
- Keep min_carriers at 2 or more; raise it for rare alleles. Read the Major line and tso_plot(plan, "major"): nothing should be *not learnable*.
- Check plan\$alleles in every plan, even without major genes.
- Declare trials, blocks, checks and covariates with fixed =; environment and trait means come automatically.

12 References

Bernardo R. (2014) Genomewide selection when major genes are known. *Crop Science* 54: 68-75.

Kang H.M., Sul J.H., Service S.K., et al. (2010) Variance component model to account for sample structure in genome-wide association studies. *Nature Genetics* 42: 348-354.

Rincet R., Laloe D., Nicolas S., et al. (2012) Maximizing the reliability of genomic selection by optimizing the calibration set of reference individuals: comparison of methods in two diverse groups of maize inbreds (*Zea mays* L.). *Genetics* 192: 715-728.

VanRaden P.M. (2008) Efficient methods to compute genomic predictions. *Journal of Dairy Science* 91: 4414-4423.